

Continuous Deployment to Tomcat

Integrating Automated Deployment into the Jenkins Pipeline

A Step-by-Step Proof of Concept (POC) Guide

Document Overview

This document builds on the earlier Continuous Integration (CI) setup by adding Continuous Deployment (CD): once a build passes, the packaged WAR artifact is automatically copied to the Tomcat server and deployed, with no manual step required. Each step below is illustrated with an actual screenshot from the setup, followed by a detailed explanation.

Scope

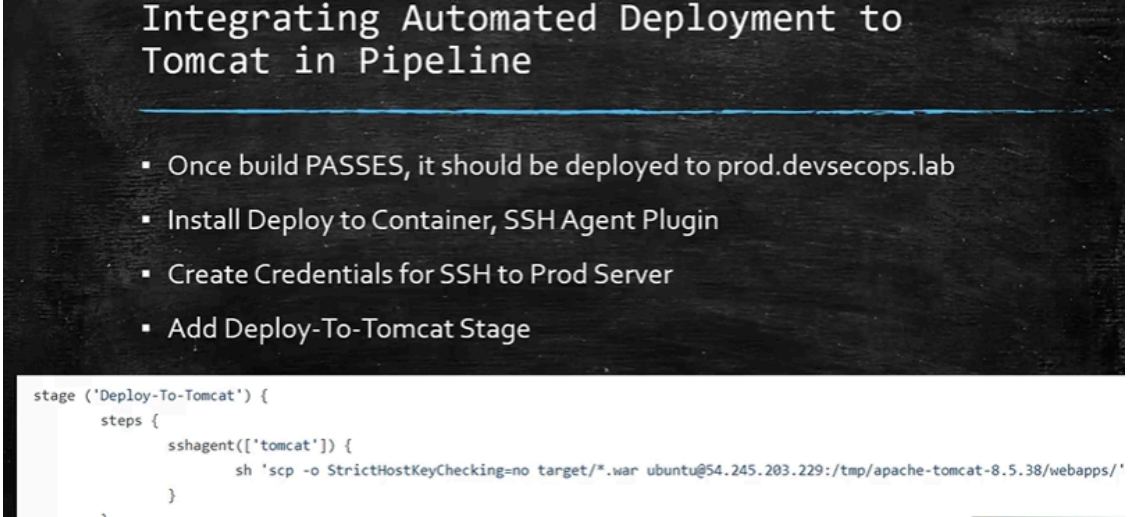
- Reference process for integrating automated deployment into a Jenkins pipeline
- Creating SSH credentials in Jenkins for the Tomcat (production) server
- Preparing the Tomcat webapps directory with the correct permissions
- Adding a Deploy-to-Tomcat stage to the Jenkinsfile using the SSH Agent plugin
- Running the pipeline and verifying a successful end-to-end deployment
- Confirming the deployed application is live and reachable

Table of Contents

- 1. Reference Slide — Integrating Automated Deployment to Tomcat
- 2. Step 1 — Open Credentials and Add New Credentials
- 3. Step 2 — Fill in the Tomcat SSH Credential Details
- 4. Step 3 — Confirm the Credential Was Created
- 5. Step 4 — Prepare Write Permissions on the Tomcat webapps Directory
- 6. Step 5 — Add the Deploy-to-Tomcat Stage in the Jenkinsfile
- 7. Step 6 — Trigger the Pipeline Build
- 8. Step 7 — Build #15 Completes Successfully
- 9. Step 8 — Application Verified Live on Tomcat
- 10. Step 9 — Reviewing the Deploy Stage in Blue Ocean
- 11. Summary and Key Takeaways

1. Reference Slide — Integrating Automated Deployment to Tomcat

The reference slide below outlines the standard process for adding automated deployment to a Jenkins pipeline: once the build passes, the artifact should be deployed to the production Tomcat server; the Deploy to Container and SSH Agent plugins need to be installed; SSH credentials for the production server must be created in Jenkins; and finally a Deploy-To-Tomcat stage is added to the pipeline. The accompanying code snippet shows the target Deploy-To-Tomcat stage structure using sshagent and scp.



Integrating Automated Deployment to Tomcat in Pipeline

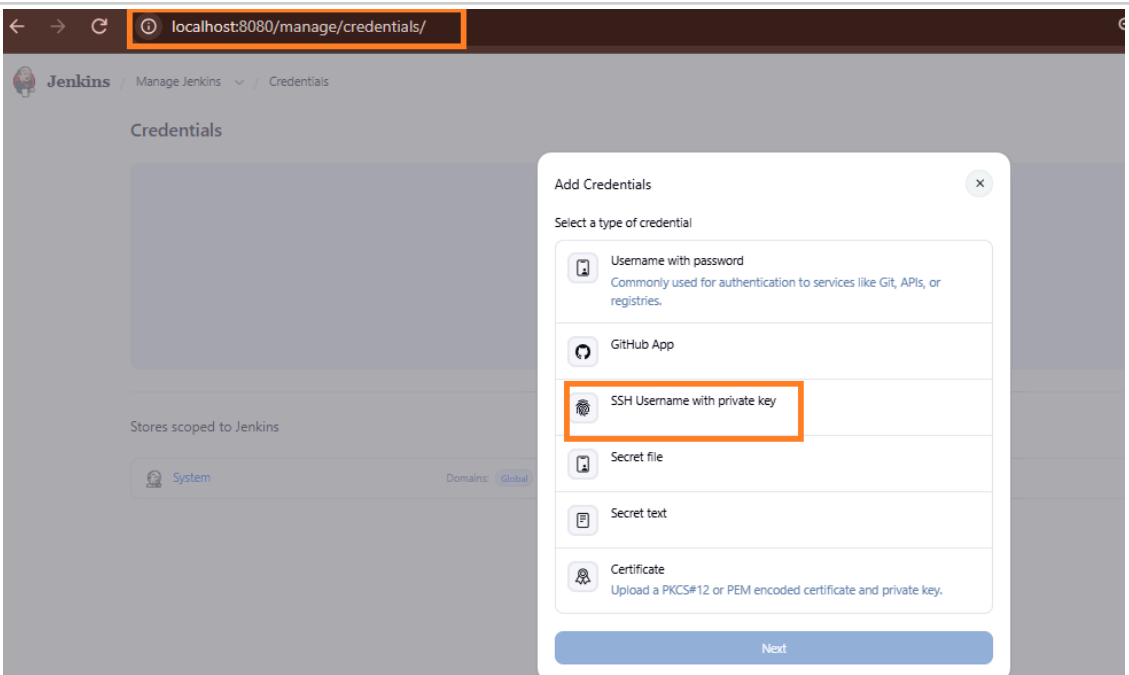
- Once build PASSES, it should be deployed to prod.devsecops.lab
- Install Deploy to Container, SSH Agent Plugin
- Create Credentials for SSH to Prod Server
- Add Deploy-To-Tomcat Stage

```
stage ('Deploy-To-Tomcat') {  
    steps {  
        sshagent(['tomcat']) {  
            sh 'scp -o StrictHostKeyChecking=no target/*.war ubuntu@54.245.203.229:/tmp/apache-tomcat-8.5.38/webapps/'  
        }  
    }  
}
```

Reference slide: steps for integrating automated deployment to Tomcat, with the target Jenkinsfile stage shown below.

2. Step 1 — Open Credentials and Add New Credentials

Deployment to the Tomcat server requires Jenkins to authenticate over SSH. The first step is to navigate to Manage Jenkins -> Credentials (<http://localhost:8080/manage/credentials/>) and open the Add Credentials dialog. Several credential types are available; SSH Username with private key is selected, since Jenkins will connect to the Tomcat instance the same way a developer would from the terminal, using the EC2 key pair rather than a password.



The screenshot shows the Jenkins web interface at localhost:8080/manage/credentials/. The 'Add Credentials' dialog is open, displaying a list of credential types. The 'SSH Username with private key' option is highlighted with an orange box. Other visible options include 'Username with password', 'GitHub App', 'Secret file', 'Secret text', and 'Certificate'. A 'Next' button is at the bottom of the dialog.

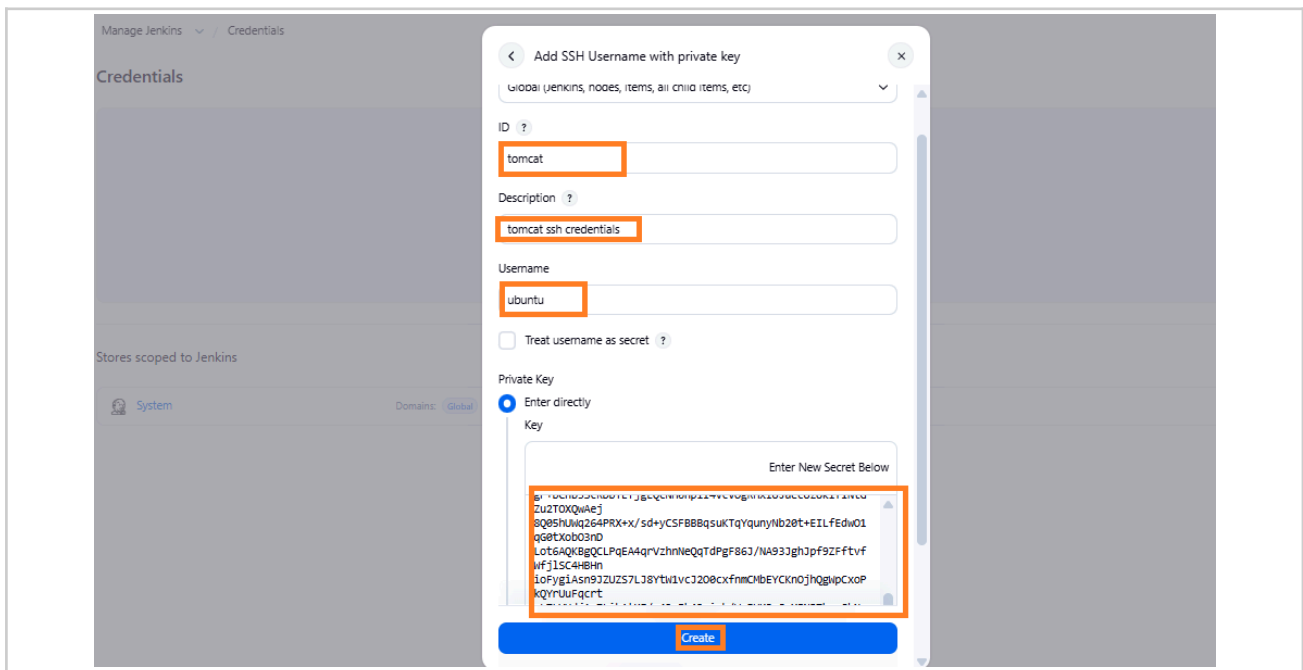
The Add Credentials dialog in Jenkins, with 'SSH Username with private key' selected as the credential type.

3. Step 2 — Fill in the Tomcat SSH Credential Details

On the credential creation form, the following values are entered:

- ID: tomcat, a short identifier used later in the Jenkinsfile to reference this credential.
- Description: tomcat ssh credentials, a human-readable label shown in the credentials list.
- Username: ubuntu, the OS user on the Tomcat EC2 instance.
- Private Key: 'Enter directly' is selected, and the full contents of the .pem key file used to SSH into the Tomcat server are pasted into the Key field.

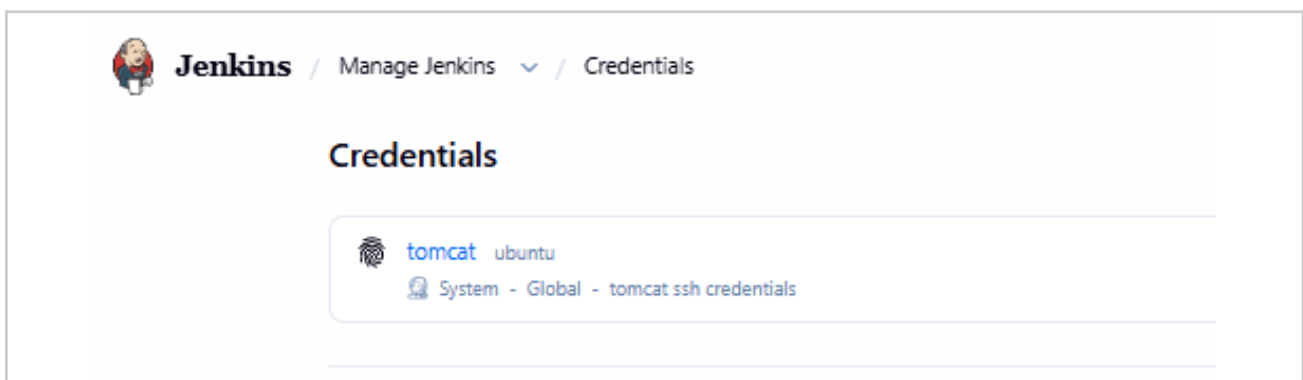
Once all fields are filled in, the Create button is clicked to save the credential.



Filling in the SSH credential form: ID 'tomcat', username 'ubuntu', and the private key pasted directly.

4. Step 3 — Confirm the Credential Was Created

After saving, the Credentials page lists the new entry: tomcat, associated with the username ubuntu, under System -> Global, with the description 'tomcat ssh credentials'. This confirms the credential is stored and available for any pipeline job to reference by its ID.



The Credentials list confirming the 'tomcat' SSH credential has been created successfully.

5. Step 4 — Prepare Write Permissions on the Tomcat webapps Directory

Before Jenkins can copy a WAR file into Tomcat's webapps directory over SCP, the ubuntu user connecting via SSH needs write access to that folder. Running `ls -la` in `/opt/tomcat` initially shows webapps with permissions `drwxr-x---`, which does not allow write access for the deploying user in this context. To resolve this, `chmod 777 webapps/` is run, after which the directory shows as `drwxrwxrwx`, writable by any user.

Caution: Setting permissions to 777 (read/write/execute for everyone) is a quick fix for a POC/demo environment, but it is not a good security practice for production. A better long-term approach is to set ownership of the webapps directory to the specific deploying user (`chown ubuntu:ubuntu webapps/`) or use a dedicated deployment group with 775 permissions, so write access is not open to all users on the system.

```
drwxr-x--- 7 ubuntu ubuntu 4096 Aug 12 19:38 webapps
drwxr-x--- 3 ubuntu ubuntu 4096 Sep 11 05:44 work
ubuntu@ip-172-31-0-121:/opt/tomcat$ chmod 777 webapps/
ubuntu@ip-172-31-0-121:/opt/tomcat$
ubuntu@ip-172-31-0-121:/opt/tomcat$
ubuntu@ip-172-31-0-121:/opt/tomcat$ ls -la
total 168
drwxr-xr-x 9 ubuntu ubuntu 4096 Sep 11 05:41 .
drwxr-xr-x 3 root root 4096 Sep 11 05:41 ..
-rw-r----- 1 ubuntu ubuntu 26700 Aug 12 19:38 BUILDING.txt
-rw-r----- 1 ubuntu ubuntu 8626 Aug 12 19:38 CONTRIBUTING.md
-rw-r----- 1 ubuntu ubuntu 57092 Aug 12 19:38 LICENSE
-rw-r----- 1 ubuntu ubuntu 2333 Aug 12 19:38 NOTICE
-rw-r----- 1 ubuntu ubuntu 3216 Aug 12 19:38 README.md
-rw-r----- 1 ubuntu ubuntu 6902 Aug 12 19:38 RELEASE-NOTES
-rw-r----- 1 ubuntu ubuntu 16543 Aug 12 19:38 RUNNING.txt
drwxr-x--- 2 ubuntu ubuntu 4096 Sep 11 05:41 bin
drwx----- 3 ubuntu ubuntu 4096 Sep 11 06:06 conf
drwxr-x--- 2 ubuntu ubuntu 4096 Sep 11 05:41 lib
drwxr-x--- 2 ubuntu ubuntu 4096 Sep 11 05:44 logs
drwxr-x--- 2 ubuntu ubuntu 4096 Sep 11 05:41 temp
drwxrwxrwx 7 ubuntu ubuntu 4096 Aug 12 19:38 webapps
drwxr-x--- 3 ubuntu ubuntu 4096 Sep 11 05:44 work
ubuntu@ip-172-31-0-121:/opt/tomcat$
```

changing permission to anyone write here it is not good but..

Changing permissions on `/opt/tomcat/webapps` to allow the deployment user to write the WAR file (777 used for this POC; noted as not ideal for production).

6. Step 5 — Add the Deploy-to-Tomcat Stage in the Jenkinsfile

With the credential and target directory ready, a third stage is added to the existing Jenkinsfile (after Initialize and Build):

```
stage('Deploy To Tomcat') {
  steps {
    sshagent(['tomcat']) {
      sh 'scp -o StrictHostKeyChecking=no target/*.war
ubuntu@13.233.2.128:/opt/tomcat/webapps/'
    }
  }
}
```

The `sshagent(['tomcat'])` block loads the previously created SSH credential (matching it by the ID 'tomcat') and makes it available to the shell step inside. The `scp` command then copies the built WAR file, matched with the wildcard `target/*.war`, to the Tomcat server's webapps directory over SSH, with host key checking disabled (`-o StrictHostKeyChecking=no`) so the first connection does not require interactive confirmation. This change is committed directly to the Jenkinsfile in the GitHub repository.

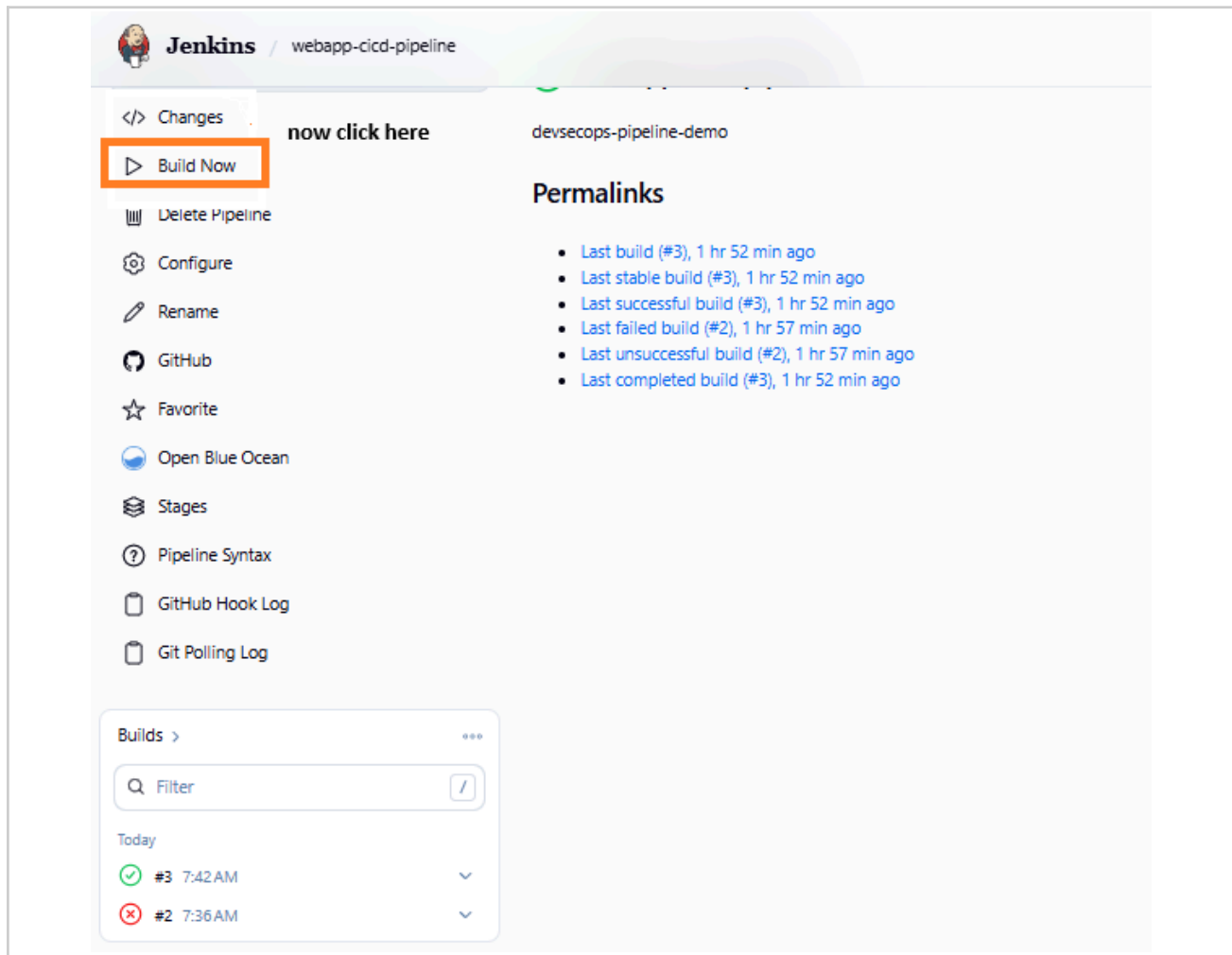
```
Code Blame 25 lines (25 loc) · 609 Bytes Raw [icon] [icon]

1  pipeline {
2      agent any
3      stages {
4          stage('Initialize') {
5              steps {
6                  sh '''
7                      echo "PATH = ${PATH}"
8                      echo "M2_HOME = ${M2_HOME}"
9                      ...
10                 }
11             }
12         stage('Build') {
13             steps {
14                 sh 'mvn clean package'
15             }
16         }
17         stage('Deploy To Tomcat') {
18             steps {
19                 sshagent(['tomcat']) {
20                     sh 'scp -o StrictHostKeyChecking=no target/*.war ubuntu@13.233.2.128:/opt/tomcat/webapps/'
21                 }
22             }
23         }
24     }
25 }
```

The completed Jenkinsfile with the Deploy To Tomcat stage added, using sshagent and scp to copy the WAR file to the Tomcat server.

7. Step 6 — Trigger the Pipeline Build

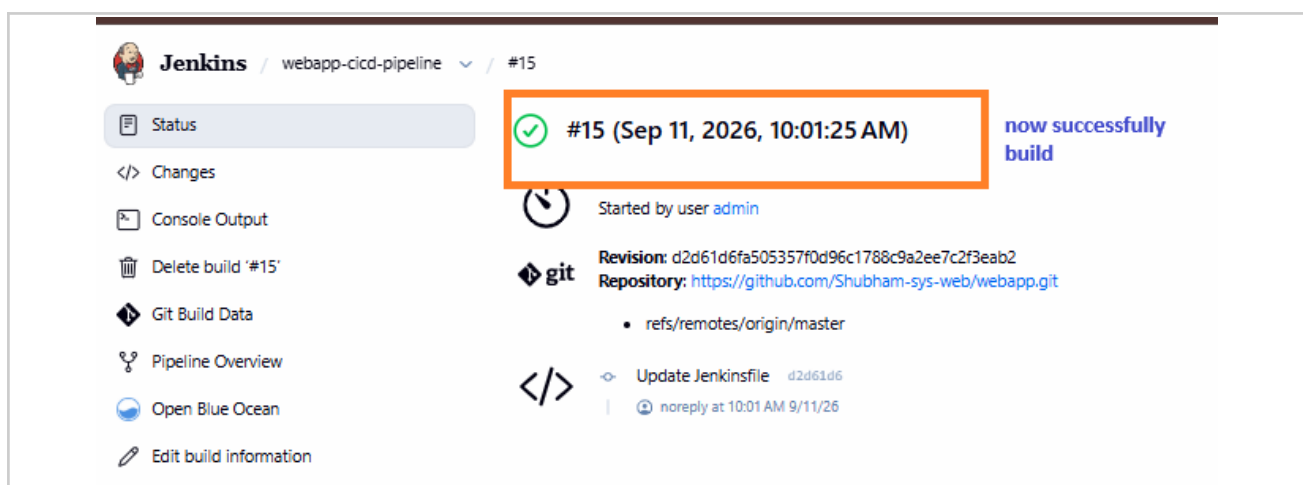
Back in the Jenkins job (webapp-cicd-pipeline), Build Now is clicked to manually trigger a new run against the updated Jenkinsfile. The job's permalinks panel shows the history of previous runs, build #3 as the last successful and last stable build at this point, confirming that this new run will be tracked as the pipeline evolves to include deployment.



Triggering a new build with 'Build Now' from the Jenkins job page, after committing the updated Jenkinsfile.

8. Step 7 — Build #15 Completes Successfully

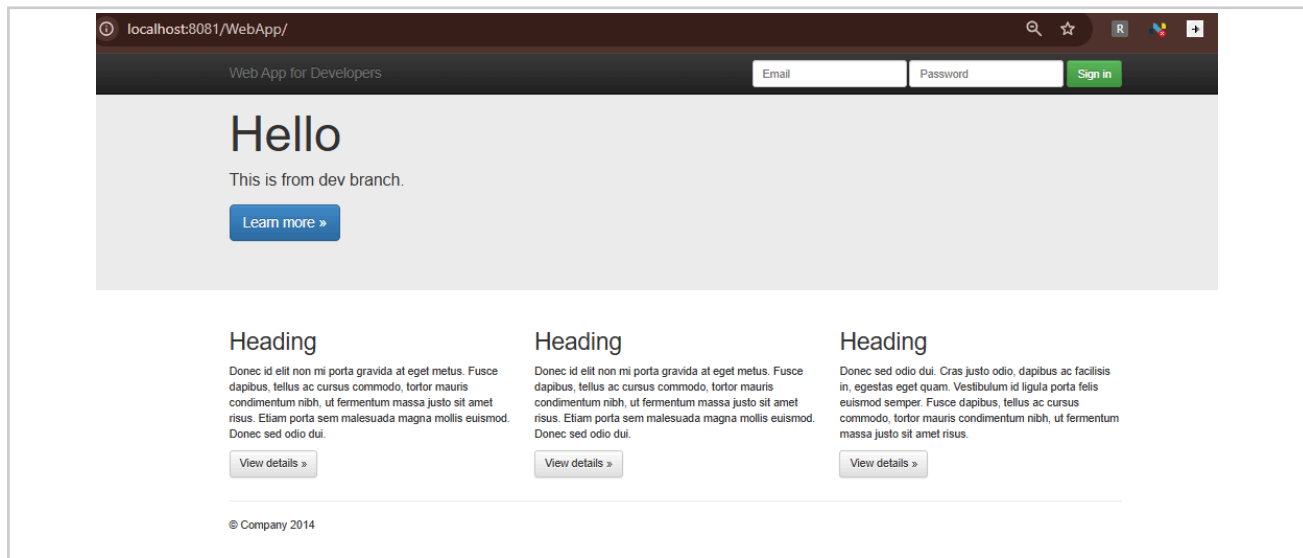
Build #15 completes with a green checkmark, confirming success. The Git Build Data section shows the exact commit that introduced the deployment stage (revision d2d61d6fa505357f0d96c1788c9a2ee7c2f3eab2, commit message 'Update Jenkinsfile', on refs/remotes/origin/master). This is the first build to include all three stages: Initialize, Build, and Deploy To Tomcat.



Build #15 completed successfully, showing the Jenkinsfile update commit that added the deployment stage.

9. Step 8 — Application Verified Live on Tomcat

To confirm the deployment worked end-to-end, the application is opened through the SSH tunnel at `http://localhost:8081/WebApp/`, matching the exact case of the WAR filename (`WebApp.war`), since Tomcat's context path is case-sensitive. The page loads successfully, displaying 'Web App for Developers' with sample content including a sign-in bar, a hero section reading 'Hello - This is from dev branch', and three content cards. This confirms the WAR file was correctly transferred and auto-deployed by Tomcat without any manual intervention.



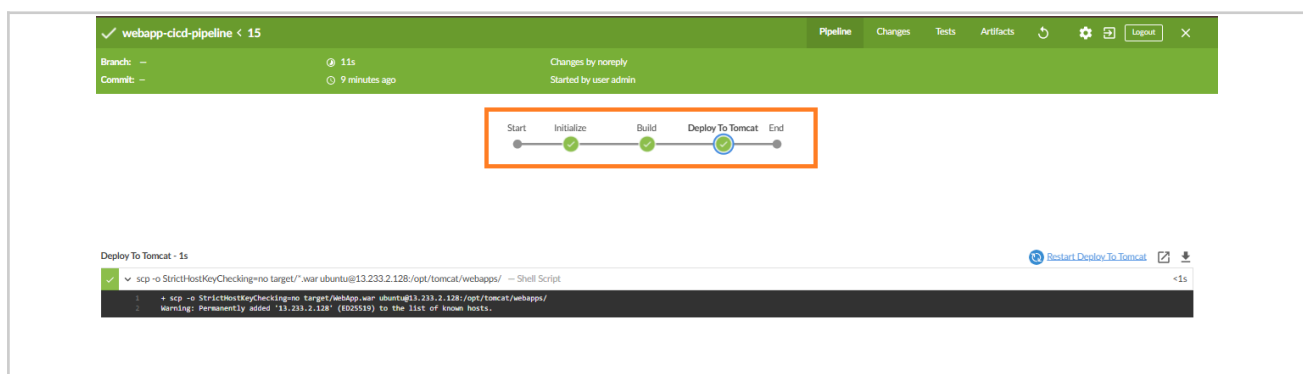
The deployed application live in the browser at `localhost:8081/WebApp/`, confirming the end-to-end pipeline worked.

10. Step 9 — Reviewing the Deploy Stage in Blue Ocean

Opening build #15 in Blue Ocean shows all four stages, Start, Initialize, Build, Deploy To Tomcat, End, marked with green checkmarks. The Deploy To Tomcat stage log shows the exact command executed:

```
scp -o StrictHostKeyChecking=no target/*.war ubuntu@13.233.2.128:/opt/tomcat/webapps/  
Warning: Permanently added '13.233.2.128' (ED25519) to the list of known hosts.
```

The stage completed in under 1 second, and the 'Warning: Permanently added...' line is expected, informational output from SSH recording the host's key on first connection, not an error. This confirms the full Continuous Integration and Continuous Deployment pipeline is working end-to-end.



Blue Ocean view of build #15: all four stages completed successfully, with the Deploy To Tomcat step log visible.

11. Summary and Key Takeaways

This POC extends the earlier CI pipeline into a full CI/CD pipeline by adding automated deployment. The key steps performed were:

- Created a dedicated SSH credential in Jenkins (ID: tomcat) using the Tomcat server's `.pem` private key, so Jenkins can authenticate without a password.

- Prepared the Tomcat webapps directory with write permissions so the deploying user can copy files into it.
- Added a Deploy To Tomcat stage to the Jenkinsfile using the SSH Agent plugin and an scp command to transfer the built WAR file.
- Triggered and verified a successful build (#15) that ran all three stages, Initialize, Build, Deploy, end-to-end.
- Confirmed the deployed application was live and reachable on the Tomcat server, with no manual deployment step required.

Note: Tomcat auto-deploys any WAR file dropped into its webapps directory within a few seconds, without needing a restart. This is what allows the pipeline to go from 'build passed' to 'application live' with a single scp command and no additional Tomcat-side action.